Dr.Kernel-8B-Better-Fusion.py

```
...

class ModelNew(nn.Module):
def __init__(self, ...):
...

def forward(self, x):
    # ConvTranspose3d using PyTorch/cuDNN
    x = self.conv_transpose(x) # x shape: [N, out_channels, D, H, W]
    ...
    if use_triton:
        # First pooling: kernel=2, stride=2, padding=0
        D_out1 = (D - 2) // 2 + 1
        H_out1 = (H - 2) // 2 + 1
        W_out1 = (W - 2) // 2 + 1

        x_pooled1 = torch.empty((N, C, D_out1, H_out1, W_out1), dtype=x.dtype, device=x.device)

    grid1 = (N, C, D_out1 * H_out1 * W_out1)
    max_pool3d_k2s2_per_channel_kernel[grid1](
    x, x_pooled1,
    N, C, D, H, W,
    D_out1, H_out1, W_out1,
    x.stride(0), x.stride(1), x.stride(2), x.stride(3), x.stride(4),
    x_pooled1.stride(0), x_pooled1.stride(1), x_pooled1.stride(2), x_pooled1.stride(3),
x_pooled1.stride(4),
    num_warps=4, num_stages=2
    )

    # Second pooling: kernel=3, stride=3, padding=0
    D_out2 = (D_out1 - 3) // 3 + 1
    H_out2 = (H_out1 - 3) // 3 + 1
    W_out2 = (W_out1 - 3) // 3 + 1

    x_pooled2 = torch.empty((N, C, D_out2, H_out2, W_out2), dtype=x.dtype, device=x.device)

    grid2 = (N, C, D_out2 * H_out2 * W_out2)
    max_pool3d_k3s3_per_channel_kernel[grid2](...)
    ...
    grid_sum = (N * D_out2 * H_out2 * W_out2,)
    sum_channels_kernel[grid_sum](
    x_pooled2, y,
    N, C, D_out2, H_out2, W_out2,
    x_pooled2.stride(0), x_pooled2.stride(1), x_pooled2.stride(2), x_pooled2.stride(3),
x_pooled2.stride(4),
    y.stride(0), y.stride(2), y.stride(3), y.stride(4),
    num_warps=4, num_stages=2
    )

    return y
```